Contents

Project Proposal: Deep Learning Analysis of Viral Molecular Mimicry in Autoimmune Myopathies

Team Members: [Your Name(s)] **Course:** CS 598 DLH - Deep Learning for Healthcare **Date:** February 2025

---

1. Executive Summary

This project aims to replicate and extend recent deep learning research on viral molecular mimicry to investigate potential links between high-incidence viral infections and autoimmune disorders, with specific focus on inclusion body myositis (IBM). We will reproduce the methodology from Ofer & Linial's 2025 study "Protein Language Models Expose Viral Immune Mimicry" and extend it by focusing on the top 10 most prevalent viruses and their associations with IBM and related inflammatory myopathies. The final deliverable will include a reproducible codebase and an interactive Streamlit web application for exploring virus-autoimmune associations.

---

2. Base Paper & Scientific Background

2.1 Primary Reference

Paper: Ofer, D., & Linial, M. (2025). Protein Language Models Expose Viral Immune Mimicry. *Viruses*, 17(9), 1199. **DOI:** Available at https://www.mdpi.com/1999-4915/17/9/1199 **Code Repository:** https://github.com/ddofer/ProteinHumVir/

2.2 Scientific Motivation

Molecular Mimicry Hypothesis: Viral proteins that structurally or functionally resemble human proteins can trigger autoimmune responses through several mechanisms: - Cross-reactive T-cell responses - B-cell activation producing auto-antibodies - Bystander activation of autoreactive immune cells

Inclusion Body Myositis (IBM): - Most common inflammatory myopathy in adults >50 years - Progressive muscle weakness, currently untreatable - Characterized by: cytotoxic T-cell infiltration, protein aggregation (TDP-43, -amyloid) - Suspected viral triggers: Retroviruses, herpesviruses (CMV, EBV), coxsackievirus - Recent evidence (Spadaro et al., 2024) links IBM to viral infections and TDP-43 pathology

2.3 Original Paper Achievements

Key Contributions: - First application of protein language models (PLMs) to viral immune mimicry detection - Achieved 99.7% ROC-AUC in distinguishing human vs. viral proteins - Identified 9.48% of viral proteins successfully evade detection through mimicry - Found specific associations with autoimmune diseases (MS, SLE) - Characterized mimicry mechanisms: IL-10 homologs, CD59 complement regulators, TCR mimicry

Datasets: - Human: 18,418 reviewed SwissProt sequences - Viral: 6,699 UniProtKB viral proteins from vertebrate hosts - Total: 25,117 protein sequences

---

3. Technical Architecture

3.1 System Overview

```
DATA ACQUISITION LAYER
```

UniProt/Swiss-Prot API (Human Prots)

ViralZone/ UniProtKB (Viral Prots)

GEO Database (GSE128470) (IBM RNAseq)

DATA PREPROCESSING LAYER

Sequence Cleaning & Validation
- Remove duplicates, invalid sequences
- Filter by length (30-1000 AA)
- UniRef50 clustering for data partitioning

Feature Engineering
- Protein embeddings extraction (ESM2, T5)
- Immunogenicity scores (NetMHCpan, IEDB)
- Structural features (disorder, hydrophobicity)

Train/Test Split (80/20 by UniRef50 cluster)

DEEP LEARNING MODEL LAYER

PRETRAINED PROTEIN LANGUAGE MODELS

MODEL 1: ESM2 (650M parameters)

- Architecture: BERT Transformer
- Layers: 33 transformer blocks
- Embedding dim: 1280

- Attention heads: 20
- Pretrained on: UniRef50 (>200M sequences)
- Max sequence length: 1024 tokens

LoRA Fine-tuning Layer

- Rank (r): 8
- Alpha ( ): 8
- Target modules: query, value projections
- Trainable params: ~0.5M (0.08% of total)

MODEL 2: ProtT5-XL (3B parameters)

- Architecture: T5 Encoder-Decoder
- Layers: 24 encoder, 24 decoder blocks
- Embedding dim: 1024
- Pretrained on: UniRef50 + BFD100
- Per-residue embeddings: 1024-dim

LoRA Fine-tuning Layer (same config as ESM2)

CLASSIFICATION HEAD

Input: [CLS] embedding (1280-d for ESM2)
OR mean-pooled (1024-d for T5)
↓
Dropout (p=0.1)
↓
Dense Layer (1280/1024 → 512)
↓
ReLU + LayerNorm
↓
Dropout (p=0.1)
↓
Dense Layer (512 → 2)
↓
Softmax → [P(human), P(viral)]

Training Configuration:
- Optimizer: AdamW (1=0.9, 2=0.999, =1e-8)
- Learning rate: 5×10 (with linear warmup)
- Batch size: 16 (gradient accumulation: 4 steps)
- Epochs: 3
- Loss: Cross-entropy with label smoothing (0.1)
- Gradient clipping: max_norm=1.0
- Mixed precision: FP16 with automatic scaling

ANALYSIS & EXTENSION LAYER

TASK 1: Viral Mimicry Scoring

For each viral protein v:
mimicry_score(v) = P(v is human | model)

High score → successful mimicry
Aggregate by: virus species, protein family

TASK 2: IBM-Specific Mimicry Analysis

Target human proteins:
- TDP-43 (TARDBP)
- Myosin heavy chain (MYH2, MYH7)
- 5'-nucleotidase 1A (NT5C1A)
- APP, -amyloid pathway proteins
- MHC Class I molecules (HLA-A, B, C)

Compute pairwise similarity:
cos_sim(embed_viral, embed_IBM_protein)

Identify hotspot regions using attention maps

TASK 3: Predictive Risk Modeling

Input features (per virus):
- Mean mimicry score
- Max mimicry score
- # proteins with score > 0.8
- Similarity to IBM proteins
- Viral family metadata

Model: Gradient Boosting (XGBoost)
→ Autoimmune risk score [0-1]

Validation: Published case-control studies

VISUALIZATION & DEPLOYMENT LAYER

STREAMLIT WEB APP

See Section 8 for detailed architecture

3.2 Model Architecture Details

3.2.1 ESM2 (Evolutionary Scale Modeling 2)

```
# Model Configuration
ESM2_CONFIG = {
    'model_name': 'esm2_t33_650M_UR50D',
    'num_layers': 33,
    'embed_dim': 1280,
    'attention_heads': 20,
    'max_positions': 1024,
    'token_dropout': True
}

# LoRA Configuration
LORA_CONFIG = {
    'r': 8,  # Low-rank dimension
    'lora_alpha': 8,  # Scaling factor
```

```
    'lora_dropout': 0.1,
    'target_modules': ['query', 'value'],  # Apply to Q,V in attention
    'bias': 'none'
}

# Classification Head
class ProteinClassifier(nn.Module):
    def __init__(self, embed_dim=1280, hidden_dim=512, num_classes=2):
        self.dropout1 = nn.Dropout(0.1)
        self.fc1 = nn.Linear(embed_dim, hidden_dim)
        self.ln1 = nn.LayerNorm(hidden_dim)
        self.dropout2 = nn.Dropout(0.1)
        self.fc2 = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        # x: [batch_size, seq_len, embed_dim]
        # Extract [CLS] token embedding
        cls_embed = x[:, 0, :]  # [batch_size, embed_dim]

        x = self.dropout1(cls_embed)
        x = F.relu(self.ln1(self.fc1(x)))
        x = self.dropout2(x)
        logits = self.fc2(x)
        return logits
```

3.2.2 Training Pipeline

```
# Loss Function
loss_fn = nn.CrossEntropyLoss(label_smoothing=0.1)

# Optimizer
optimizer = AdamW(
    model.parameters(),
    lr=5e-4,
    betas=(0.9, 0.999),
    eps=1e-8,
    weight_decay=0.01
)

# Learning Rate Scheduler
num_training_steps = len(train_loader) * num_epochs
num_warmup_steps = num_training_steps // 10
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=num_warmup_steps,
    num_training_steps=num_training_steps
)
```

```
# Training Loop (pseudo-code)
for epoch in range(3):
    for batch in train_loader:
        with autocast():  # Mixed precision
            embeddings = esm_model(batch['sequences'])
            logits = classifier(embeddings)
            loss = loss_fn(logits, batch['labels'])

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer)
        scaler.update()
        scheduler.step()
```

3.3 Extended Analysis Methodology

3.3.1 Mimicry Score Computation

```
def compute_mimicry_score(viral_protein_seq, model):
    """
    Compute how well a viral protein mimics human proteins.

    Returns:
        mimicry_score: float [0-1], P(protein is human | model)
    """
    embeddings = model.encode(viral_protein_seq)
    logits = model.classify(embeddings)
    probs = F.softmax(logits, dim=-1)

    # probs[0] = P(human), probs[1] = P(viral)
    mimicry_score = probs[0].item()

    return mimicry_score
```

3.3.2 IBM-Specific Pairwise Analysis

```
def analyze_ibm_mimicry(viral_proteome, ibm_proteins, model):
    """
    Compute embedding similarity between viral and IBM-related proteins.
    """
    results = []

    for viral_prot in viral_proteome:
        v_embed = model.encode(viral_prot, output_hidden_states=True)

        for ibm_prot in ibm_proteins:
            h_embed = model.encode(ibm_prot, output_hidden_states=True)
```

```
            # Cosine similarity in embedding space
            similarity = cosine_similarity(v_embed, h_embed)

            # Attention-weighted similarity (identify hotspot regions)
            attention_scores = extract_attention(model, viral_prot, ibm_prot)
            hotspots = find_high_attention_regions(attention_scores, threshold=0.7)

            results.append({
                'viral_protein': viral_prot.id,
                'ibm_protein': ibm_prot.name,
                'embedding_similarity': similarity,
                'hotspot_regions': hotspots
            })

    return pd.DataFrame(results)
```

---

4. Target Viruses & Datasets

4.1 Top 10 High-Incidence Viruses

Virus	Family	Global Prevalence	IBM Association Evidence
Epstein-Barr Virus (EBV)	Herpesviridae	>90% adults	Moderate (T-cell activation)
Cytomegalovirus (CMV)	Herpesviridae	50-80% adults	Strong (found in muscle biopsies)
Influenza A/B	Orthomyxoviridae	Seasonal, high	Weak (post-viral myositis)
Herpes Simplex Virus 1/2	Herpesviridae	60-95% adults	Weak
Varicella-Zoster Virus (VZV)	Herpesviridae	>90% adults	Weak
Human Immunodeficiency Virus	Retroviridae	0.7% global	Moderate (HIV myopathy)
Hepatitis C Virus (HCV)	Flaviviridae	1% global	Moderate (autoimmune overlap)

Virus	Family	Global Prevalence	IBM Association Evidence
SARS-CoV-2	Coronaviridae	>70% exposed	Emerging (post-COVID myositis)
Human Papillomavirus (HPV)	Papillomaviridae	≥80% adults	Very weak
Coxsackievirus B	Picornaviridae	Common	Moderate (viral myositis)

4.2 IBM-Related Protein Targets

Protein	UniProt ID	Role in IBM
TDP-43	Q13148	Aggregates in muscle fibers
Myosin Heavy Chain 2	Q9UKX2	Muscle-specific, potential antigen
Myosin Heavy Chain 7	P12883	Cardiac/skeletal muscle
Cytoplasmic 5'-nucleotidase 1A	Q9BXI3	Auto-antibody target
Amyloid Precursor Protein (APP)	P05067	Aggregates in IBM
HLA-A*01:01	P30443	MHC I upregulation in IBM
HLA-B*08:01	P30450	Associated with IBM risk
-Synuclein	P37840	Protein aggregation

4.3 Data Sources

```
DATA_SOURCES = {
    'human_proteins': {
        'source': 'UniProt/Swiss-Prot',
        'url': 'https://ftp.uniprot.org/pub/databases/uniprot/current_release/knowledgebase/co
        'filter': 'reviewed:yes AND organism:"Homo sapiens (Human) [9606]"',
        'expected_count': '~20,000 reviewed sequences'
    },
    'viral_proteins': {
        'source': 'UniProtKB + ViralZone',
        'viruses': {
            'EBV': 'taxonomy:10376',
            'CMV': 'taxonomy:10359',
            'Influenza_A': 'taxonomy:11320',
            'HSV1': 'taxonomy:10298',
            'HIV1': 'taxonomy:11676',
            # ... etc
        },
        'filter': 'host:"Homo sapiens [9606]"'
    },
    'ibm_transcriptomics': {
        'source': 'GEO (Gene Expression Omnibus)',
        'datasets': [
```

```
            'GSE128470',  # IBM vs healthy muscle biopsies
            'GSE39454',   # Inflammatory myopathies
        ],
        'purpose': 'Identify upregulated proteins in IBM for focused analysis'
    },
    'validation_data': {
        'seroprevalence': 'Published literature (PubMed search)',
        'case_control': 'IBM patient registries (Euromyositis, NIH)',
        'clinical_associations': 'Manual curation from recent reviews'
    }
}
```

---

5. Experimental Design

5.1 Reproduction Phase (Weeks 1-3)

Objective: Validate that we can replicate original paper's results

Steps: 1. Set up computational environment (PyTorch, HuggingFace Transformers, PEFT) 2. Download and preprocess original datasets (25,117 sequences) 3. Implement ESM2 + LoRA fine-tuning pipeline 4. Train model and validate performance metrics 5. **Success Criteria:** Achieve >99% ROC-AUC on human vs. viral classification

Expected Outputs: - Trained model checkpoint - Performance metrics (ROC-AUC, precision, recall, F1) - Confusion matrix analysis - Misclassification error analysis

5.2 Extension Phase (Weeks 4-7)

Objective: Apply methodology to virus-IBM hypothesis

Experiment 1: Focused Viral Panel Analysis

```
EXPERIMENT_1 = {
    'name': 'Top-10 Virus Mimicry Profiling',
    'input': '10 target virus proteomes (~3,000 viral proteins)',
    'method': 'Compute mimicry scores using trained model',
    'analysis': [
        'Rank viruses by mean/max mimicry score',
        'Identify which viral proteins are top mimics',
        'Compare mimicry patterns across virus families'
    ],
    'visualization': [
        'Heatmap: virus × mimicry score distribution',
        'Violin plots: mimicry score by virus family',
        'Network graph: high-mimicry viral proteins'
    ]
}
```

Experiment 2: IBM Protein Similarity Analysis

```
EXPERIMENT_2 = {
    'name': 'Viral Mimicry of IBM-Associated Proteins',
    'input': '10 viruses × 8 IBM target proteins',
    'method': 'Embedding cosine similarity + attention analysis',
    'analysis': [
        'Which viruses show highest similarity to TDP-43?',
        'Are there shared mimicry patterns across myosins?',
        'Do HLA molecules show mimicry (MHC I upregulation)?'
    ],
    'statistical_tests': [
        'Permutation test: observed similarity vs. random',
        'FDR correction for multiple comparisons'
    ]
}
```

Experiment 3: Predictive Risk Modeling

```
EXPERIMENT_3 = {
    'name': 'Autoimmune Risk Prediction from Mimicry Features',
    'input': 'Aggregated mimicry features + clinical evidence labels',
    'method': 'XGBoost classifier with SHAP explainability',
    'labels': {
        'Strong evidence': ['CMV', 'Coxsackievirus'],
        'Moderate evidence': ['EBV', 'HIV', 'HCV'],
        'Weak/No evidence': ['Influenza', 'HPV', 'HSV']
    },
    'features': [
        'mean_mimicry_score',
        'max_mimicry_score',
        'num_high_mimics (>0.8)',
        'similarity_to_TDP43',
        'similarity_to_myosins',
        'viral_family_encoded'
    ],
    'validation': '5-fold cross-validation + leave-one-virus-out'
}
```

5.3 Computational Resources

Hardware Requirements: - GPU: NVIDIA A100 (40GB) or V100 (32GB) for ESM2 inference - CPU: 16+ cores for data preprocessing - RAM: 64GB minimum - Storage: 500GB (models, datasets, results)

Software Stack:

```
environment:
  python: 3.10
  pytorch: 2.0+
```

```
transformers: 4.30+
peft: 0.4+  # Parameter-Efficient Fine-Tuning
biopython: 1.81
fair-esm: 2.0
scikit-learn: 1.3
xgboost: 1.7
streamlit: 1.25+
plotly: 5.15+
pandas: 2.0+
```

Estimated Compute Time: - Data preprocessing: 4-6 hours - Model training (3 epochs): 8-12 hours (with A100) - Inference on 25K sequences: 2-3 hours - Extension experiments: 10-15 hours - **Total:** ~30-40 GPU hours

---

6. Evaluation Metrics

6.1 Reproduction Validation

Metric	Original Paper	Our Target
ROC-AUC	99.7%	>99.0%
Precision (Human)	~98%	>97%
Recall (Human)	~99%	>98%
F1-Score	~98.5%	>97.5%
Viral Misclassification Rate	9.48%	8-11%

6.2 Extension Evaluation

Biological Validation: - Correlation with published IBM-virus association strength (Spearman's) - Agreement with known molecular mimicry cases (e.g., EBV-MS)

Statistical Significance: - p-values for virus-IBM protein similarities (permutation tests) - Effect sizes (Cohen's d) for mimicry score differences

Predictive Performance: - Risk model AUC-ROC on held-out virus - SHAP feature importance alignment with biological knowledge

---

7. Expected Outcomes & Impact

7.1 Primary Deliverables

1. **Reproduced Model:** Validated ESM2-LoRA classifier with >99% accuracy
2. **Virus Mimicry Atlas:** Comprehensive scoring of 10 viruses across IBM proteins
3. **Risk Prediction Model:** XGBoost model predicting autoimmune associations
4. **Interactive Web App:** Streamlit dashboard for exploring results (see Section 8)
5. **Research Report:** 8-10 page paper documenting findings

7.2 Expected Findings

Hypothesis 1: CMV and EBV will show highest mimicry scores and strongest TDP-43 similarity **Hypothesis 2:** Herpesviruses (EBV, CMV, HSV) will cluster together in mimicry patterns **Hypothesis 3:** Mimicry scores will correlate with published IBM case-control associations

7.3 Broader Impact

- **Clinical:** Guide targeted viral testing in IBM patients
- **Therapeutic:** Identify antiviral intervention targets
- **Methodological:** Demonstrate PLM utility for autoimmune research
- **Reproducibility:** Fully open-source pipeline for future studies

---

8. Streamlit Web Application Architecture

8.1 Application Overview

The Streamlit application will serve as an interactive platform for exploring viral mimicry results, enabling both technical and non-technical users to investigate virus-autoimmune associations.

```
                    STREAMLIT WEB APPLICATION
                     (streamlit_app.py)



                          MAIN NAVIGATION

[Home] [Explore Viruses] [IBM Analysis] [Model Inference]
[About] [Documentation]



PAGE 1: HOME & PROJECT OVERVIEW

• Project motivation and scientific background
• Key findings summary
• Interactive infographic: molecular mimicry mechanism
• Quick stats: total proteins analyzed, top mimics



PAGE 2: VIRUS EXPLORATION DASHBOARD


  SIDEBAR FILTERS
    Select Viruses:
      Epstein-Barr Virus (EBV)
```

```
    Cytomegalovirus (CMV)
    Influenza A
  [... all 10 viruses]

Mimicry Score Range: [0.0        1.0]
Viral Family: [All ]
```

VISUALIZATION PANEL

TAB 1: Mimicry Score Distribution
- Interactive violin plot (Plotly)
- Box plot overlays showing quartiles
- Hover: protein name, score, annotation

TAB 2: Heatmap View
- Rows: Viral proteins
- Columns: Viruses
- Color: Mimicry score [0=blue, 1=red]
- Click to drill down

TAB 3: Network Graph
- Nodes: Viral proteins (size = mimicry score)
- Edges: Sequence similarity
- Color by virus family
- Interactive zoom/pan (Plotly/Cytoscape.js)

TAB 4: Comparison Stats
- Bar chart: Mean mimicry by virus
- Statistical tests: Kruskal-Wallis p-values
- Effect sizes

DATA TABLE
Virus | Protein | Mimicry Score | Length | Family
[Sortable, filterable, downloadable as CSV]

PAGE 3: IBM-SPECIFIC ANALYSIS

SELECT IBM TARGET PROTEIN
Radio buttons:
  TDP-43 Myosin Heavy Chain 2

NT5C1A APP
[... all 8 proteins]

VISUALIZATION: TOP VIRAL MIMICS
• Horizontal bar chart: Top 20 viral proteins
• Bars colored by virus
• X-axis: Embedding cosine similarity
• Click to see sequence alignment

ATTENTION HEATMAP
• Rows: Human protein residues
• Columns: Viral protein residues
• Color: Attention weight (darker = higher)
• Highlights "hotspot" regions

SEQUENCE ALIGNMENT VIEWER
Human: MSSKKRGRKG...VTHVIVPYTQAPAS...
|||| | || ...|||||| ||
Viral: MSSRGRGRKG...VTHVIAPYTQAPGS...
[Highlighted conserved regions]

VIRUS RANKING TABLE
Rank | Virus | Mean Similarity | Max Similarity |
| | to [selected] | Protein |
[Download results as PDF report]

PAGE 4: LIVE MODEL INFERENCE

INPUT PROTEIN SEQUENCE

>my_protein
MKTIIALSYIFCLVFADYKDDDDK...

[Paste FASTA sequence here]

OR Upload File: [Choose File] (.fasta, .fa)

[Run Prediction]

PREDICTION RESULTS

Classification: [VIRAL] / [HUMAN]

Confidence: 85.3%

Mimicry Score: 0.147 (Low mimicry)

Interpretation:
"This sequence is predicted as VIRAL with high confidence. Low mimicry score suggests it is unlikely to trigger autoimmune responses."

Attention Visualization

- Residue-level importance scores
- Highlight key regions contributing to prediction

Disclaimer: Research tool only. Not for clinical use.

PAGE 5: AUTOIMMUNE RISK PREDICTOR

SELECT VIRUS FOR RISK ASSESSMENT
Dropdown: [EBV]

PREDICTED AUTOIMMUNE RISK

RISK GAUGE

72% High Risk

Evidence Level: MODERATE-STRONG

Feature Importance (SHAP Values)

Mean mimicry score	0.45
Similarity to TDP-43	0.38
Max mimicry score	0.25
Viral family (Herpes)	0.15
# high-mimicry proteins	0.08

Supporting Evidence

- CMV DNA found in 45% IBM biopsies (Ref: [1])
- Case-control OR: 2.3 (95% CI: 1.1-4.8, p=0.02)
- T-cell reactivity to CMV pp65 in IBM (Ref: [2])

PAGE 6: DOCUMENTATION & METHODS

- Model architecture diagram
- Training procedure and hyperparameters
- Dataset descriptions and statistics
- Code repository link (GitHub)
- Citation information
- Download full report (PDF)
- FAQ section

8.2 Technical Implementation

8.2.1 Application Structure

```
streamlit_app/
  app.py                    # Main entry point
  pages/
     1_ _Home.py
     2_ _Virus_Explorer.py
     3_ _IBM_Analysis.py
     4_ _Model_Inference.py
     5_ _Risk_Predictor.py
     6_ _Documentation.py
  utils/
     data_loader.py       # Load precomputed results
     model_wrapper.py     # Load ESM2 model for inference
     visualizations.py    # Plotly/altair chart functions
     sequence_utils.py    # FASTA parsing, validation
  data/
     mimicry_scores.csv   # Precomputed viral mimicry scores
     ibm_similarities.csv # IBM protein similarity matrix
```

```
    risk_predictions.csv # XGBoost predictions per virus
    metadata.json        # Virus/protein annotations
models/
    esm2_lora_finetuned.pt  # Model checkpoint (optional)
    risk_model.pkl           # XGBoost model
assets/
    logo.png
    molecular_mimicry.svg
    architecture_diagram.png
requirements.txt
README.md
```

8.2.2 Key Streamlit Components

```python
# app.py - Main configuration
import streamlit as st

st.set_page_config(
    page_title="Viral Mimicry & Autoimmunity Explorer",
    page_icon=" ",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Custom CSS for styling
st.markdown("""
    <style>
    .main-header {
        font-size: 3rem;
        color: #1E88E5;
        text-align: center;
    }
    .metric-card {
        background-color: #f0f2f6;
        padding: 20px;
        border-radius: 10px;
        box-shadow: 2px 2px 5px rgba(0,0,0,0.1);
    }
    </style>
""", unsafe_allow_html=True)
```

```python
# pages/2_ _Virus_Explorer.py
import streamlit as st
import plotly.express as px
from utils.data_loader import load_mimicry_scores

st.title("  Virus Mimicry Explorer")
```

```
# Sidebar filters
with st.sidebar:
    st.header("Filters")
    selected_viruses = st.multiselect(
        "Select Viruses",
        options=["EBV", "CMV", "Influenza A", ...],
        default=["EBV", "CMV"]
    )

    score_range = st.slider(
        "Mimicry Score Range",
        min_value=0.0,
        max_value=1.0,
        value=(0.0, 1.0)
    )

# Load and filter data
df = load_mimicry_scores()
df_filtered = df[
    (df['virus'].isin(selected_viruses)) &
    (df['mimicry_score'].between(*score_range))
]

# Visualization tabs
tab1, tab2, tab3 = st.tabs(["Distribution", "Heatmap", "Network"])

with tab1:
    fig = px.violin(
        df_filtered,
        x='virus',
        y='mimicry_score',
        color='virus',
        box=True,
        points='all',
        hover_data=['protein_name', 'protein_id']
    )
    st.plotly_chart(fig, use_container_width=True)

with tab2:
    # Pivot for heatmap
    heatmap_data = df_filtered.pivot_table(
        index='protein_name',
        columns='virus',
        values='mimicry_score'
    )
    fig = px.imshow(
        heatmap_data,
        color_continuous_scale='RdBu_r',
```

```
        aspect='auto'
    )
    st.plotly_chart(fig, use_container_width=True)

# Data table with download
st.subheader("Raw Data")
st.dataframe(df_filtered, use_container_width=True)

st.download_button(
    label="  Download CSV",
    data=df_filtered.to_csv(index=False),
    file_name="viral_mimicry_filtered.csv",
    mime="text/csv"
)
```

```
# pages/4_ _Model_Inference.py
import streamlit as st
from utils.model_wrapper import ViralClassifier
from utils.sequence_utils import parse_fasta, validate_sequence

st.title("  Live Model Inference")

# Initialize model (cached)
@st.cache_resource
def load_model():
    return ViralClassifier(model_path="models/esm2_lora_finetuned.pt")

model = load_model()

# Input methods
input_method = st.radio("Input Method", ["Paste Sequence", "Upload FASTA"])

if input_method == "Paste Sequence":
    sequence = st.text_area(
        "Paste protein sequence (FASTA format)",
        height=150,
        placeholder=">my_protein\nMKTIIALSYIFCLVFADYKDDDDK..."
    )
else:
    uploaded_file = st.file_uploader("Upload FASTA file", type=['fasta', 'fa'])
    if uploaded_file:
        sequence = uploaded_file.read().decode()

if st.button("  Run Prediction"):
    if sequence:
        try:
            # Parse and validate
            seq_id, seq_str = parse_fasta(sequence)
```

```
            validate_sequence(seq_str)

            # Run inference
            with st.spinner("Running model..."):
                prediction = model.predict(seq_str)

            # Display results
            col1, col2 = st.columns(2)

            with col1:
                st.metric(
                    "Prediction",
                    prediction['class'],
                    delta=f"{prediction['confidence']:.1%} confidence"
                )

            with col2:
                st.metric(
                    "Mimicry Score",
                    f"{prediction['mimicry_score']:.3f}",
                    delta="Low" if prediction['mimicry_score'] < 0.5 else "High"
                )

            # Interpretation
            st.info(prediction['interpretation'])

            # Attention visualization
            st.subheader("Attention Heatmap")
            fig = plot_attention(prediction['attention_weights'], seq_str)
            st.plotly_chart(fig)

        except Exception as e:
            st.error(f"Error: {str(e)}")
    else:
        st.warning("Please provide a sequence")
```

8.2.3 Deployment Strategy Option 1: Streamlit Community Cloud (Free)

```
# .streamlit/config.toml
[theme]
primaryColor = "#1E88E5"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F0F2F6"
textColor = "#262730"
font = "sans serif"

[server]
maxUploadSize = 50  # MB for FASTA uploads
```

```
enableCORS = false
```

Option 2: Docker Containerization

```
# Dockerfile
FROM python:3.10-slim

WORKDIR /app

# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application
COPY streamlit_app/ ./streamlit_app/
COPY models/ ./models/
COPY data/ ./data/

EXPOSE 8501

CMD ["streamlit", "run", "streamlit_app/app.py", \
     "--server.port=8501", \
     "--server.address=0.0.0.0"]
```

Deployment Commands:

```
# Local testing
streamlit run streamlit_app/app.py

# Deploy to Streamlit Cloud
# 1. Push code to GitHub
# 2. Connect repo at share.streamlit.io
# 3. Select app.py as entry point

# Docker deployment
docker build -t viral-mimicry-app .
docker run -p 8501:8501 viral-mimicry-app

# Access at http://localhost:8501
```

8.2.4 Performance Optimizations

```
# Use caching for expensive operations
@st.cache_data(ttl=3600)
def load_precomputed_results():
    """Load CSV files (cached for 1 hour)"""
    return pd.read_csv("data/mimicry_scores.csv")

@st.cache_resource
def load_deep_learning_model():
```

```
    """Load PyTorch model (cached permanently)"""
    model = ESM2Classifier()
    model.load_state_dict(torch.load("models/checkpoint.pt"))
    return model

# Lazy loading for large visualizations
with st.spinner("Generating network graph..."):
    if st.button("Show Network"):
        fig = create_network_graph(df)
        st.plotly_chart(fig)
```

8.3 User Experience Features

Accessibility: - Screen reader compatible with `alt` text for all visuals - Keyboard navigation support - High contrast mode toggle - Font size adjustment

Educational Components: - Tooltips explaining technical terms (e.g., "What is mimicry score?") - Interactive tutorial on first visit - Glossary page with definitions - Video demonstration of key features

Export Options: - Download filtered datasets as CSV/Excel - Export visualizations as PNG/SVG/PDF - Generate PDF report summarizing selected virus - Citation export in BibTeX/RIS formats

---

9. Timeline & Milestones

Week	Phase	Tasks	Deliverables
1-2	**Setup & Reproduction**	• Environment setup • Download datasets • Implement ESM2 pipeline • Train baseline model	• Working codebase • Trained model checkpoint • Reproduction validation report
3-4	**Data Curation**	• Collect 10 virus proteomes • Curate IBM protein targets • Download IBM transcriptomics • Preprocess & validate	• Curated dataset (3K+ sequences) • Data statistics report

Week	Phase	Tasks	Deliverables
5-6	**Extension Experiments**	• Compute mimicry scores • IBM similarity analysis • Statistical testing • Visualizations	• Results CSV files • Initial figures • Preliminary findings
7	**Risk Modeling**	• Feature engineering • Train XGBoost • SHAP analysis • Validation	• Risk prediction model • Feature importance plots
8	**Streamlit Development**	• Build web app • Integrate visualizations • Deploy to cloud • User testing	• Live web application • Public URL
9	**Analysis & Writing**	• Interpret results • Create figures for report • Draft manuscript • Peer review within team	• Draft report (80% complete)
10	**Finalization**	• Revise report • Record presentation video • Finalize code documentation • Submit deliverables	• Final report (PDF) • 5-min video • GitHub repo • Streamlit app

---

10. Potential Challenges & Mitigation Strategies

Challenge	Risk Level	Mitigation
GPU access limitations	Medium	• Use Google Colab Pro or university HPC • Implement gradient checkpointing • Use smaller ESM2 variant (150M params) if needed

Challenge	Risk Level	Mitigation
Difficulty reproducing original results	Medium	• Contact authors for clarifications • Test multiple hyperparameter configurations • Document deviations transparently
Limited IBM patient data	High	• Focus on published datasets (GEO) • Use meta-analysis from literature • Clearly state limitations in report
Computational time exceeds estimates	Low	• Parallelize inference across GPUs • Precompute embeddings once • Use caching strategies
Biological interpretation challenges	Medium	• Consult with domain experts • Cross-reference multiple databases • Clearly distinguish correlation vs. causation
Streamlit deployment issues	Low	• Test locally first • Use Docker for consistency • Have backup static dashboard (Plotly Dash)

---

11. Ethical Considerations

Responsible AI: - Clearly label predictions as research-grade, not clinical-grade - Provide confidence intervals and uncertainty estimates - Avoid overstating causal claims (mimicry proven causation)

Data Privacy: - Use only publicly available, de-identified datasets - No patient-level data in web application - Comply with data use agreements (GEO, UniProt)

Transparency: - Full code and data availability (GitHub, Zenodo) - Document all preprocessing steps - Report negative results if hypotheses fail

Broader Impact Statement: - Acknowledge potential for misinterpretation - Emphasize need for experimental validation - Discuss implications for vaccine development (check mimicry)

---

12. References

Primary Paper: 1. Ofer, D., & Linial, M. (2025). Protein Language Models Expose Viral Immune Mimicry. *Viruses*, 17(9), 1199.

Supporting Literature: 2. McLeish, E., et al. (2024). From data to diagnosis: how machine learning is revolutionizing biomarker discovery in idiopathic inflammatory myopathies. *Rheumatology*, 63(1), 20-29. 3. Spadaro, M., et al. (2024). Inclusion body myositis, viral infections, and TDP-43: a narrative review. *Clinical and Experimental Medicine*, 24(1), 78. 4. Lundberg, I. E., et

al. (2021). European League Against Rheumatism/American College of Rheumatology classification criteria for adult and juvenile idiopathic inflammatory myopathies and their major subgroups. *Arthritis & Rheumatology*, 69(12), 2271-2282.

Technical Resources: 5. Lin, Z., et al. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123-1130. 6. Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*.

---

13. Team Contributions (if applicable)

[To be filled based on team composition]

Member 1: - Model implementation and training - Reproduction validation - Code documentation

Member 2: - Data curation and preprocessing - Statistical analysis - IBM-specific experiments

Member 3: - Streamlit application development - Visualization design - Final report writing

---

14. Conclusion

This project combines cutting-edge deep learning (protein language models) with clinically relevant questions (viral triggers of autoimmune disease). By replicating a high-quality recent publication and extending it to a focused hypothesis (virus-IBM associations), we will demonstrate both technical competency and biological insight. The interactive Streamlit application will make our findings accessible to a broad audience, enhancing the project's impact and educational value.

Key Innovations: 1. First application of PLMs to IBM-specific autoimmune research 2. Systematic profiling of 10 high-incidence viruses 3. Integration of mimicry scores with predictive risk modeling 4. Interactive web platform for hypothesis exploration

We anticipate that this work will generate testable hypotheses for experimental virologists and provide a reproducible framework for future autoimmune-infectious disease studies.

Appendices:

A. Detailed hyperparameter configurations B. Data preprocessing scripts C. Statistical test specifications D. Streamlit code snippets E. Generative AI usage log (ChatGPT/Claude interactions)

[To be completed during project execution]